

1 -

J lexemas-tokens.java/Main.java

1import java.util.LinkedHashMap;
2import java.util.Map;
3
4public class Main {
5 static final String ALUNO = "Leonardo Camilotti Moreno";
6 static final String RA = "24015988-2";
7
8 static final String ENTRADA = "resultado = somaAnterior - valor / 1000;";
9
10 private static final Map<Character, String> SIMBOLOS = new LinkedHashMap<>();
11
12 static {
13 SIMBOLOS.put('=', "ATRIBUICAO");
14 SIMBOLOS.put('+', "SOMA");
15 SIMBOLOS.put('-', "SUBTRACAO");
16 }
17 }
18

ConsoleI/OAI Agent

ALUNO: Leonardo Camilotti Moreno | RA: 24015988-2
ENTRADA: resultado = somaAnterior - valor / 1000;

LEXEMA TOKEN

resultado IDENTIFICADOR
= ATRIBUICAO
somaAnterior IDENTIFICADOR
- SUBTRACAO
valor IDENTIFICADOR
/ DIVISAO
1000 INTEIRO
; PONTO_E_VIRGULA

2 -

J 02-classes-reservadas/Main.java

1import java.util.Arrays;
2import java.util.HashSet;
3import java.util.Set;
4
5public class Main {
6 static final String ALUNO = "Leonardo Camilotti Moreno";
7 static final String RA = "24015988-2";
8
9 static final String IDENTIFICADOR_PERSONALIZADO = "nota_lm";
10
11 static final Set<String> PALAVRAS_RESERVADAS = new HashSet<>(
12 Arrays.asList("int", "if", "else", "while", "return")
13);
14
15 public static void main(String[] args) {
16 System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
17 executarTeste("TESTE A - entrada compacta",
18 "int total=valor+10;");
19 executarTeste("TESTE B - espaços e comentário",
20 "int total = valor + 10; // este comentario sera ignorado");
21 executarTeste("TESTE C - reservadas e identificadores",
22 "int inteiro = 0; while (inteiro < intValor) inteiro = inteiro + 1;");
23 executarTeste("TESTE C2 - identificador personalizado",
24 "int " + IDENTIFICADOR_PERSONALIZADO + " = 10;");
25 executarTeste("TESTE D - caractere inválido",
26 "int valor# = 1;");
27 }
28 }
29

ConsoleI/OAI Agent1.0s

inteiro LETRA IDENTIFICADOR
+ SIMBOLO SOMA
1 DIGITO INTEIRO
; SIMBOLO PONTO_E_VIRGULA

=== TESTE C2 - identificador personalizado ===
ENTRADA: int nota_lm = 10;
LEXEMA CLASSE TOKEN

int LETRA PALAVRA_RESERVADA
nota_lm LETRA IDENTIFICADOR
= SIMBOLO ATRIBUICAO
10 DIGITO INTEIRO
; SIMBOLO PONTO_E_VIRGULA

=== TESTE D - caractere inválido ===
ENTRADA: int valor# = 1;
LEXEMA CLASSE TOKEN

int LETRA PALAVRA_RESERVADA
valor LETRA IDENTIFICADOR
DESCONHECIDO ERRO_LEXICO
= SIMBOLO ATRIBUICAO
1 DIGITO INTEIRO
; SIMBOLO PONTO_E_VIRGULA

3 -

J 03-diagnosticos/Main.java

1public class Main {
2 static final String ALUNO = "Leonardo Camilotti Moreno";
3 static final String RA = "24015988-2";
4
5 public static void main(String[] args) {
6 System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
7
8 int total = 10 + 20 // CORRIGIR 1: acrescente o ponto e virgula
9
10 if (total > 20 { // CORRIGIR 2: feche corretamente o parenteses
11 System.out.println("Programa corrigido!"); // CORRIGIR 3: feche o texto entre aspas
12 System.out.println("Resultado: " + total);
13 }
14 }
15 }
16

ConsoleI/OAI AgentExit 1

Main.java:8: error: ';' expected
int total = 10 + 20 // CORRIGIR 1:
acrescente o ponto e virgula
 ^
Main.java:10: error: ')' expected
if (total > 20 { // CORRIGIR 2: feche
corretamente o parenteses
 ^
Main.java:11: error: unclosed string literal
System.out.println("Programa
corrigido!"); // CORRIGIR 3: feche o texto
entre aspas
 ^
3 errors
Debug this error

J 03-diagnosticos/Main.java

ConsoleI/OAI Agent

ALUNO: Leonardo Camilotti Moreno | RA: 24015988-2
Programa corrigido!
Resultado: 30

```
1 public class Main {
2     static final String ALUNO = "Leonardo Camilotti Moreno";
3     static final String RA = "24015988-2";
4
5     public static void main(String[] args) {
6         System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
7
8         int total = 10 + 20; // CORRIGIR 1: acrescente o ponto e virgula
9
10        if (total > 20) { // CORRIGIR 2: feche corretamente o parenteses
11            System.out.println("Programa corrigido!"); // CORRIGIR 3: feche o texto entre aspas
12            System.out.println("Resultado: " + total);
13        }
14    }
15 }
16
```

4 -

J 04-parser-descendente/Main.java

ConsoleI/OAI Agent940 ms

SAINDO de <term> | lookahead = +
Encontrado operador +
Novo lookahead: 3
ENTRANDO em <term> | lookahead = 3
ENTRANDO em <factor> | lookahead = 3

Numero reconhecido: 3
Novo lookahead:)
SAINDO de <factor> | lookahead =)
SAINDO de <term> | lookahead =)
SAINDO de <expr> | lookahead =)
Novo lookahead: *
Fechando parenteses
SAINDO de <factor> | lookahead = *
Encontrado operador *
Novo lookahead: 4
ENTRANDO em <factor> | lookahead = 4
Numero reconhecido: 4
Novo lookahead: \$FIM\$
SAINDO de <factor> | lookahead = \$FIM\$
SAINDO de <term> | lookahead = \$FIM\$
SAINDO de <expr> | lookahead = \$FIM\$

ENTRADA ACEITA
RESULTADO DA EXPRESSAO: 20

```
1 public class Main {
2     static final String ALUNO = "Leonardo Camilotti Moreno";
3     static final String RA = "24015988-2";
4
5     static final String ENTRADA = "(2 + 3) * 4";
6     static final boolean DEMONSTRAR_RECURSAO_ESQUERDA = false;
7
8     public static void main(String[] args) {
9         System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
10        System.out.println("ENTRADA: " + ENTRADA);
11        System.out.println();
12
13        if (DEMONSTRAR_RECURSAO_ESQUERDA) {
14            demonstrarRecursoEsquerda(0);
15            System.out.println();
16        }
17
18        try {
19            Parser parser = new Parser(ENTRADA);
20            int resultado = parser.analisar();
21            System.out.println();
22            System.out.println("ENTRADA ACEITA");
23            System.out.println("RESULTADO DA EXPRESSAO: " + resultado);
24        } catch (ErroSintatico erro) {
25            System.out.println();
26            System.out.println("ERRO SINTATICO: " + erro.getMessage());
27        }
28    }
29 }
```

5 -

J 05-parser-ascendente/Main.java

ConsoleI/OAI Agent1.3s

ALUNO: Leonardo Camilotti Moreno | RA: 24015988-2
ENTRADA: id+id*id

PILHAENTRADAAAO

\$ 0id+id*id\$SHIFT id -> estado 5
\$ 0 id 5+id*id\$REDUCE F -> id
\$ 0 F 3+id*id\$REDUCE T -> F
\$ 0 T 2+id*id\$REDUCE E -> T
\$ 0 E 1+id*id\$SHIFT + -> estado 6
\$ 0 E 1 + 6id*id\$SHIFT id -> estado 5
\$ 0 E 1 + 6 id 5*id\$REDUCE F -> id
\$ 0 E 1 + 6 F 3*id\$REDUCE T -> F
\$ 0 E 1 + 6 T 9*id\$SHIFT * -> estado 7
\$ 0 E 1 + 6 T 9 * 7id\$SHIFT id -> estado 5
\$ 0 E 1 + 6 T 9 * 7 id 5\$REDUCE F -> id
\$ 0 E 1 + 6 T 9 * 7 F 10\$REDUCE T -> T * F
\$ 0 E 1 + 6 T 9\$REDUCE E -> E + T
\$ 0 E 1\$ACCEPT

```
1 import java.util.ArrayList;
2 import java.util.HashMap;
3 import java.util.List;
4 import java.util.Map;
5
6 public class Main {
7     static final String ALUNO = "Leonardo Camilotti Moreno";
8     static final String RA = "24015988-2";
9
10    static final String ENTRADA_VALIDA = "id+id*id";
11    static final String ENTRADA_INVALIDA = "id*id";
12
13    // Na primeira execucao, use ENTRADA_VALIDA. Depois troque por ENTRADA_INVALIDA
14    static final String ENTRADA = ENTRADA_VALIDA;
15
16    static final Map<Integer, Map<String, String>> ACTION = new HashMap<>();
17    static final Map<Integer, Map<String, Integer>> GOTO = new HashMap<>();
18    static final Map<Integer, Regra> REGRAS = new HashMap<>();
19
20    static {
21        // Gramatica:
22        // 1) E -> E + T 2) E -> T
```